

[DOC] medc

Documentación Técnica · AutoDocs AI · 09/06/2026

12	27	7	3
Archivos	Funciones	Clases	Endpoints

Score de Calidad: 68/100

Documentación del Proyecto

Generado automáticamente por **AutoDocs AI** — 2026-06-09 21:45 UTC

Resumen Ejecutivo

Este proyecto está desarrollado principalmente en **Python** y cuenta con 12 archivos fuente, 27 funciones, 7 clases y 3 endpoints de API documentados. El score de calidad del código es **68/100**.

Stack Tecnológico

Lenguaje	Cantidad
YAML	1 archivos
Python	11 archivos

Estadísticas del Proyecto

Métrica	Valor
Total de archivos	12
Funciones	27
Clases	7
Endpoints API	3
Score de calidad	68/100
Funciones documentadas	0 / 27

2. Documentación de Arquitectura y Diseño

2.1 Estructura de Directorios

```
proyecto/  
■■■ (raíz)/  
■■■ controllers/  
■■■ entorno/  
■■■ models/  
■■■ repository/  
■■■ services/
```

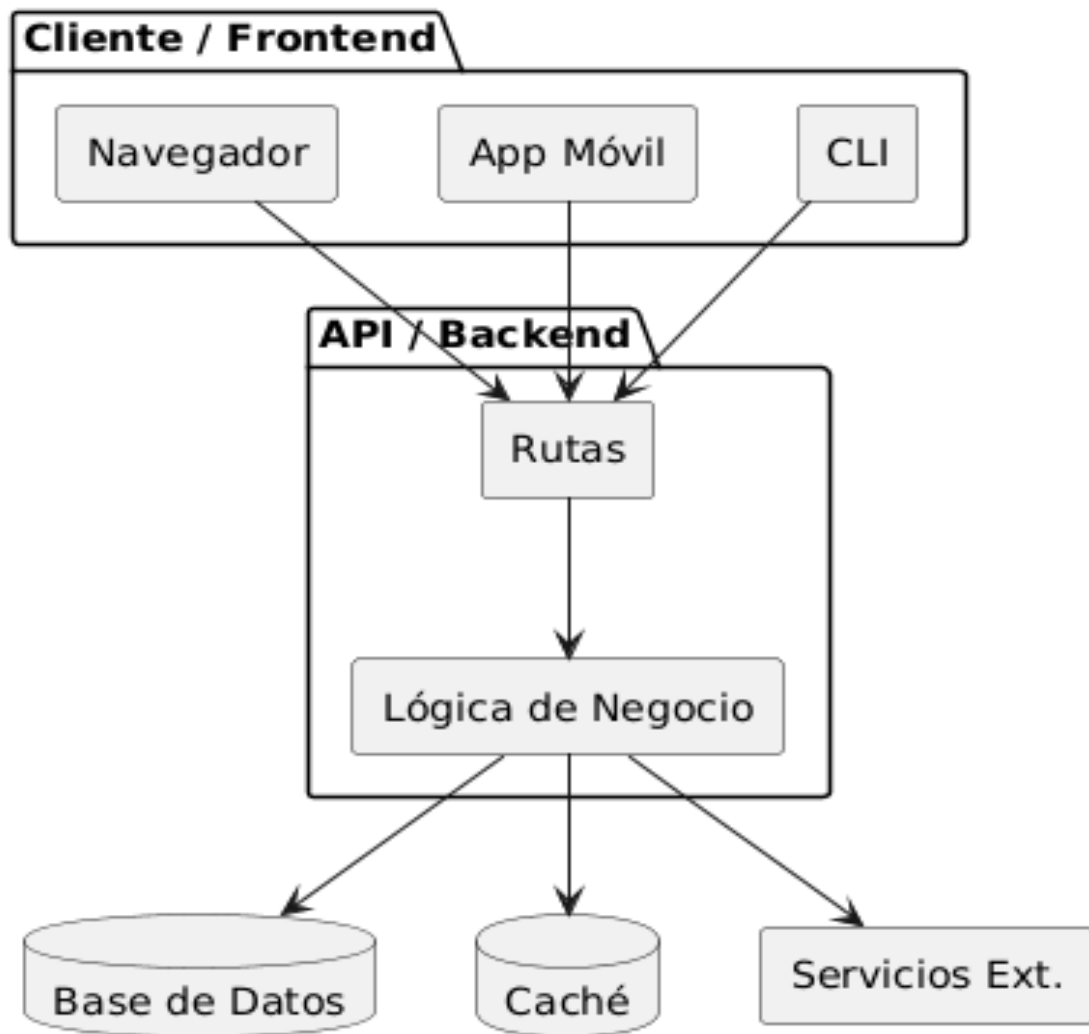
2.2 Distribución por Capas

- `(raíz)/` — 5 archivos [.env(1), .gitignore(1), .yaml(1), .py(1), .txt(1)]
- `controllers/` — 2 archivos [.py(2)]
- `entorno/` — 2 archivos [.py(2)]
- `models/` — 2 archivos [.py(2)]
- `repository/` — 2 archivos [.py(2)]
- `services/` — 2 archivos [.py(2)]

2.3 Distribución de Lenguajes

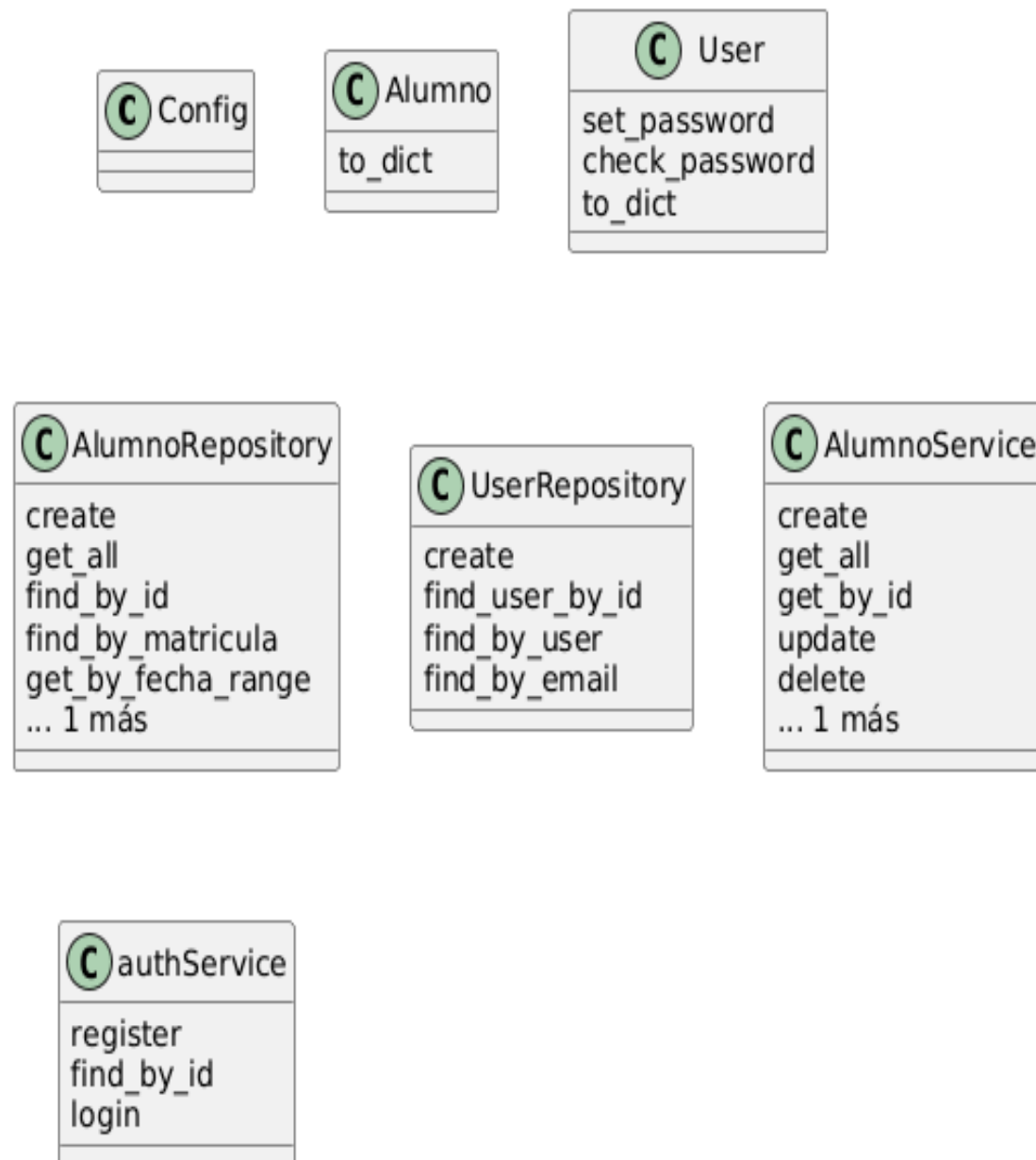
- Python: 11 archivos (92%)
- YAML: 1 archivos (8%)

2.4 Diagrama de Infraestructura



2.5 Diagrama de Casos de Uso

2.6 Diagrama de Clases



■■ **Clases con información incompleta:** Las siguientes clases aparecen sin métodos ni atributos documentados: `Config`. Para que el diagrama se visualice correctamente, agrega métodos, propiedades o docstrings a estas clases en tu código fuente y regenera la documentación.

2.7 Patrones Detectados

- **API REST:** 3 endpoints detectados (Flask)

2.8 Insights de Arquitectura (IA)

Análisis del Proyecto

- **Patrones arquitectónicos probables:** Monolito, debido a la relativamente baja cantidad de endpoints y clases.

- **Buenas prácticas detectadas:** Uso de Python como lenguaje principal y YAML para configuración.
 - **Áreas de mejora sugeridas:** Aumentar la modularidad y separación de concerns para mejorar la mantenibilidad.
 - **Complejidad del proyecto:** Baja-Moderada, con 27 funciones y 7 clases.
 - **Recomendaciones de escalabilidad:** Considerar la implementación de un patrón de arquitectura más escalable como Microservicios si el proyecto crece en complejidad o tamaño. También, considerar el uso de frameworks como Flask o Django para mejorar la estructura y el manejo de endpoints.
-

3. Documentación Técnica de Código

3.1 Guía de Configuración del Entorno de Desarrollo

```
# 1. Clonar el repositorio
git clone <url-del-repositorio>
cd <nombre-del-proyecto>

# 2. Crear entorno virtual (recomendado para Python)
python -m venv venv
source venv/bin/activate # Linux/Mac
venv\Scripts\activate   # Windows

# 3. Instalar dependencias
pip install -r requirements.txt

# 4. Configurar variables de entorno
cp .env.example .env
# Editar .env con tus valores locales

# 5. Ejecutar en modo desarrollo
python app.py # o npm run dev
```

3.2 Documentación de API

Total de endpoints detectados: 3

Módulo: controllers\AuthController.py

[POST] POST /register

- **Framework:** Flask
- **Archivo:** controllers\AuthController.py

Descripción del endpoint de registro

Propósito: El propósito de este endpoint es permitir a los usuarios crear una cuenta en un sistema o aplicación. Piensa en ello como cuando te registras en una tienda en línea o en una red social.

Funcionalidad: Cuando se utiliza este endpoint, se envía una solicitud al servidor para crear una nueva cuenta de usuario. El servidor entonces verifica la información proporcionada y crea la cuenta si todo está

correcto.

Información necesaria: Para que este endpoint funcione, necesitas proporcionar algunos datos básicos, como:

- Un nombre de usuario (o correo electrónico)
- Una contraseña segura
- Posiblemente otros datos personales, como nombre y apellido

Resultado: Si la solicitud es exitosa, el servidor devuelve un mensaje que indica que la cuenta ha sido creada con éxito. Esto puede incluir un código de confirmación o un enlace para activar la cuenta.

Uso en la vida real: En la vida real, este endpoint se utiliza en situaciones como:

- Crear una cuenta en una tienda en línea para hacer compras
- Registrarse en una red social para conectarse con amigos y familiares
- Crear una cuenta en un servicio de streaming para acceder a contenido exclusivo
- Registrarse en un foro o comunidad en línea para participar en discusiones y compartir ideas

En resumen, este endpoint es

Parámetros esperados:

- `body` (JSON, requerido): Cuerpo de la solicitud
- `Authorization` (Header, requerido): Bearer JWT token

Códigos de respuesta:

- `200` — Operación exitosa
- `400` — Solicitud incorrecta
- `401` — No autorizado
- `404` — Recurso no encontrado
- `500` — Error interno del servidor

[GET] `GET /register/<int:id>`

- **Framework:** Flask
- **Archivo:** `controllers\AuthController.py`

Descripción del Endpoint

Propósito: El propósito de este endpoint es obtener información sobre un registro específico. Imagina que tienes una base de datos con muchos registros, y necesitas encontrar uno en particular.

Funcionalidad: Este endpoint se utiliza para recuperar información sobre un registro que ya existe en la base de datos. Es como buscar un libro en una biblioteca, pero en lugar de buscar por título o autor, buscas por un número de identificación único (el "id").

Información necesaria: Para que este endpoint funcione, necesitas proporcionar el número de identificación único del registro que deseas obtener. Este número se conoce como "id" y se incluye en la dirección web del endpoint (la "ruta").

Resultado: Cuando se llama a este endpoint, devuelve la información asociada con el registro que tiene el id proporcionado. Es como recibir el libro que buscas en la biblioteca, con toda la información que contiene.

Uso en la vida real: En la vida real, este tipo de endpoint se utiliza en aplicaciones web para recuperar información sobre un usuario, un producto o un servicio. Por ejemplo:

- En un sitio web de compras en línea, podrías utilizar este endpoint para obtener la información de un producto específico, como su descripción, precio y características.
- En una aplicación de redes sociales, podrías utilizar este endpoint para obtener la información de un usuario específico

Parámetros esperados:

- **Authorization** (Header, requerido): Bearer JWT token

Códigos de respuesta:

- **200** — Operación exitosa
- **400** — Solicitud incorrecta
- **401** — No autorizado
- **404** — Recurso no encontrado
- **500** — Error interno del servidor

[POST] **POST** /login

- **Framework:** Flask
- **Archivo:** `controllers\AuthController.py`

Descripción del Endpoint de API: /login

1. Propósito del Endpoint

El propósito de este endpoint es permitir a los usuarios acceder a un sistema o aplicación de manera segura. En otras palabras, es como una puerta de entrada que verifica quién eres y si tienes permiso para entrar.

2. Funcionalidad

Cuando se utiliza este endpoint, envía una solicitud al servidor para verificar las credenciales de acceso (como nombre de usuario y contraseña). Luego, el servidor comprueba si las credenciales son correctas y, si lo son, permite al usuario acceder al sistema o aplicación.

3. Información Necesaria

Para que este endpoint funcione, se necesita proporcionar dos piezas de información básica:

- Un nombre de usuario (como un correo electrónico o un nombre de cuenta)
- Una contraseña (una serie de caracteres secretos que solo el usuario conoce)

Estos datos se envían al servidor para que pueda verificar si son correctos.

4. Resultado

Si las credenciales son correctas, el servidor devuelve un mensaje de éxito, que puede incluir un token o una clave de acceso que permite al usuario acceder a la aplicación o sistema de manera segura. Si las credenciales son incorrectas, el servidor devuelve un mensaje de error, indicando que no se pudo acceder.

5. Uso en

Parámetros esperados:

- **body** (JSON, requerido): Cuerpo de la solicitud
- **Authorization** (Header, requerido): Bearer JWT token

Códigos de respuesta:

- 200 — Operación exitosa
- 400 — Solicitud incorrecta
- 401 — No autorizado
- 404 — Recurso no encontrado
- 500 — Error interno del servidor

3.3 Estado de Comentarios y Docstrings

Total de funciones: 27

- [OK] Documentadas: 0 (0%)
- [X] Sin documentar: 27 (100%)

Visualización de documentación:



Funciones que requieren documentación:

- `create_app()` en `app.py`
- `register()` en `controllers\AuthController.py`
- `get_user_by_id(id)` en `controllers\AuthController.py`
- `login()` en `controllers\AuthController.py`

- `to_dict(self)` en `models\alumnos.py`
- ... y 22 más

3.4 Planes de Prueba (Testing)

No se detectaron pruebas automatizadas. Se recomienda implementar:

```
# Ejemplo de estructura de tests recomendada
import pytest

class TestEndpoints:
    def test_get_returns_200(self, client):
        response = client.get('/api/endpoint')
        assert response.status_code == 200

    def test_post_creates_resource(self, client):
        data = {'name': 'test'}
        response = client.post('/api/endpoint', json=data)
        assert response.status_code == 201

    def test_unauthorized_returns_401(self, client):
        response = client.get('/api/protected')
        assert response.status_code == 401
```

Escenarios de prueba recomendados:

Escenario	Tipo	Prioridad
POST /register — respuesta 200	Integración	Alta
POST /register — sin auth (401)	Seguridad	Alta
GET /register/<int:id> — respuesta 200	Integración	Alta
GET /register/<int:id> — sin auth (401)	Seguridad	Alta
POST /login — respuesta 200	Integración	Alta
POST /login — sin auth (401)	Seguridad	Alta

4. Documentación de Clases

Total: **7 clases**

Config

- **Archivo:** `entorno\config.py`
- **Línea:** 6
- **Hereda de:** `object`
- **Métodos:** 0

- Documentada: [X] No

Introducción a la Clase Config

La clase Config es una parte fundamental en un proyecto de programación en Python. A continuación, te explicaré de manera sencilla qué es, qué representa, qué puede hacer y por qué es importante.

1. Propósito de la clase

Imagina que estás organizando una fiesta y necesitas tener todo en orden. La clase Config es como un cuaderno de notas donde se almacenan las instrucciones y configuraciones básicas para que todo funcione correctamente en el proyecto. Su propósito es almacenar y proporcionar información importante para que el proyecto se ejecute de manera efectiva.

2. Qué representa esta clase en el sistema

La clase Config se puede comparar con un panel de control en un aeropuerto. Al igual que el panel de control muestra la información necesaria para que los pilotos y el personal del aeropuerto puedan tomar decisiones informadas, la clase Config proporciona la información necesaria para que el proyecto se ejecute de manera correcta. En este caso, el panel de control es la clase Config, y la información que muestra es la configuración del proyecto.

3. Qué puede hacer esta clase

La clase Config puede hacer varias cosas, como:

- Almacenar información importante sobre el proyecto, como la ubicación de los archivos y las conexiones a bases de datos.
- Proporcionar acceso a

Alumno

- Archivo: `models\alumnos.py`
- Línea: 4
- Hereda de: `db.Model`
- Métodos: 1
- Documentada: [X] No

Clase Alumno: Una forma de organizar información sobre estudiantes

1. Propósito de la clase: La clase Alumno es como un contenedor donde se almacena información sobre cada estudiante en un sistema. Su propósito es ayudar a mantener organizada y accesible la información sobre los alumnos.

2. Qué representa esta clase: Imagina un archivo de estudiantes en una escuela. Cada archivo tiene el nombre del estudiante, su edad, su curso, etc. La clase Alumno es como uno de esos archivos, pero en lugar de ser físico, es digital y se encuentra dentro del sistema.

3. Qué puede hacer esta clase: La clase Alumno puede almacenar y proporcionar información sobre cada estudiante, como su nombre, edad, curso, etc. También tiene una función especial llamada "to_dict"

que permite convertir la información del estudiante en un formato fácil de entender y compartir.

4. Para qué se usa en la vida real: En la vida real, esta clase se podría usar en un sistema de gestión escolar para almacenar y mantener actualizada la información de los estudiantes. Por ejemplo, un profesor podría usar el sistema para ver la lista de estudiantes en su clase, o un administrador podría usarlo para generar informes sobre el desempeño de los estudiantes.

5. Importancia de esta clase en el sistema: La clase Alumno es importante porque permite al

Métodos:

- `to_dict()`

User

- **Archivo:** `models\user.py`
- **Línea:** 4
- **Hereda de:** `db.Model`
- **Métodos:** 3
- **Documentada:** [X] No

Introducción a la Clase User

La Clase User es un componente fundamental en nuestro proyecto de Python. En términos simples, su propósito es representar a un usuario dentro del sistema. Piensa en ella como una tarjeta de identificación que almacena información sobre una persona que utiliza nuestro sistema.

Representación en el sistema

Imagina que estás en un hotel y necesitas registrarte para obtener una habitación. La recepción te pide tu nombre, dirección, correo electrónico y una forma de identificación. De manera similar, la Clase User es como una tarjeta de registro para los usuarios de nuestro sistema. Almacena información básica sobre cada usuario, como su nombre, correo electrónico y contraseña.

Funcionalidades principales

La Clase User puede hacer varias cosas importantes:

- **Establecer una contraseña:** Es como crear una combinación para una caja fuerte. La Clase User te permite establecer una contraseña segura para tu cuenta.
- **Verificar la contraseña:** Es como intentar abrir la caja fuerte con la combinación correcta. La Clase User puede verificar si la contraseña que ingresaste es la correcta.
- **Convertir información a un formato legible:** Es como tomar la información de tu tarjeta de identificación y ponerla en un papel para que sea fácil de leer. La Clase User puede tomar la información del usuario y convertirla en un

Métodos:

- `set_password()`

- `check_password()`
 - `to_dict()`
-

AlumnoRepository

- **Archivo:** `repository\AlumnoRepository.py`
- **Línea:** 5
- **Hereda de:** `object`
- **Métodos:** 6
- **Documentada:** [X] No

Descripción de la clase AlumnoRepository

1. Propósito de la clase:

La clase `AlumnoRepository` es como un administrador de información de estudiantes. Su función principal es ayudar a gestionar y organizar la información de los alumnos de manera eficiente.

2. Representación en el sistema:

Imagina un archivo grande donde se almacena toda la información de los estudiantes. La clase `AlumnoRepository` es como un bibliotecario que se encarga de ordenar, buscar y actualizar la información de los estudiantes en ese archivo. De esta manera, es fácil encontrar y gestionar la información de cada alumno.

3. Funcionalidades principales:

La clase `AlumnoRepository` puede hacer varias cosas útiles:

- * Crear nuevos registros de estudiantes (`create`)
- * Mostrar todos los registros de estudiantes (`get_all`)
- * Buscar un estudiante específico por su ID o matrícula (`find_by_id`, `find_by_matricula`)
- * Mostrar los registros de estudiantes dentro de un rango de fechas específico (`get_by_fecha_range`)
- * Eliminar un registro de estudiante (`delete`)

Estas funcionalidades permiten que la información de los estudiantes sea gestionada de manera efectiva y organizada.

4. Uso en la vida real:

En la vida real, una clase como `AlumnoRepository` se puede utilizar en sistemas de gestión de escuelas, universidades o institutos. Por

Métodos:

- `create()`
- `get_all()`
- `find_by_id()`
- `find_by_matricula()`

- `get_by_fecha_range()`
 - `delete()`
-

UserRepository

- **Archivo:** `repository\userRepository.py`
- **Línea:** 4
- **Hereda de:** `object`
- **Métodos:** 4
- **Documentada:** [X] No

Introducción a la clase UserRepository

La clase UserRepository es una parte fundamental de un sistema de gestión de usuarios. Imagina que estás en una biblioteca y necesitas encontrar un libro específico. La biblioteca tiene un catálogo que te permite buscar libros por título, autor o tema. De manera similar, la clase UserRepository es como un catálogo de usuarios que te permite buscar y gestionar información de los usuarios en un sistema.

Propósito de la clase

El propósito de la clase UserRepository es almacenar y gestionar la información de los usuarios en un sistema. Esto incluye crear nuevos usuarios, buscar usuarios existentes y recuperar información sobre ellos.

Qué representa esta clase en el sistema

La clase UserRepository representa un "almacén" de información de usuarios. Al igual que un archivo en una oficina, donde se guardan documentos importantes, la clase UserRepository almacena la información de los usuarios de manera organizada y segura.

Qué puede hacer esta clase

La clase UserRepository puede realizar las siguientes funciones:

- Crear nuevos usuarios (`create`): es como agregar un nuevo libro a la biblioteca.
- Buscar un usuario por su ID (`find_user_by_id`): es como buscar un libro por su número de catálogo.
- Buscar un usuario por su nombre de usuario (`find_by_user`): es como buscar un libro por su título.
- Buscar un usuario por su correo electrónico (`find_by_email`):

Métodos:

- `create()`
 - `find_user_by_id()`
 - `find_by_user()`
 - `find_by_email()`
-

- **Archivo:** `services\alumnoService.py`
- **Línea:** 5
- **Hereda de:** `object`
- **Métodos:** 6
- **Documentada:** [X] No

Introducción a la Clase AlumnoService

1. Propósito de la clase:

La clase AlumnoService es como un asistente que ayuda a gestionar información sobre alumnos en un sistema. Su propósito es simplificar y organizar las tareas relacionadas con la información de los alumnos, como crear, leer, actualizar y eliminar registros.

2. Representación en el sistema:

Imagina un archivo de estudiantes en una escuela. Esta clase es como el administrador de ese archivo, responsable de mantener actualizada y organizada la información de cada estudiante. Al igual que un bibliotecario que ayuda a encontrar libros, esta clase ayuda a encontrar y gestionar la información de los alumnos de manera eficiente.

3. Funcionalidades principales:

La clase AlumnoService puede hacer varias cosas:

- **Crear un nuevo registro de alumno:** Es como agregar un nuevo estudiante al archivo de la escuela.
- **Ver todos los registros de alumnos:** Es como revisar todo el archivo de estudiantes para ver quiénes están registrados.
- **Buscar un alumno específico:** Es como encontrar un estudiante en particular en el archivo.
- **Actualizar la información de un alumno:** Es como corregir o actualizar los detalles de un estudiante en el archivo.
- **Eliminar un registro de alumno:** Es como retirar el registro de un estudiante que ya no está en la escuela.
- ****Buscar alumnos**

Métodos:

- `create()`
- `get_all()`
- `get_by_id()`
- `update()`
- `delete()`
- `get_by_fecha_range()`

- **Archivo:** `services\authService.py`
- **Línea:** 4
- **Hereda de:** `object`
- **Métodos:** 3
- **Documentada:** [X] No

Descripción de la clase `authService`

1. Propósito de la clase:

La clase `authService` es como un guardián de seguridad que ayuda a proteger el acceso a un sistema o aplicación. Su objetivo es asegurarse de que solo las personas autorizadas puedan entrar y utilizar los recursos del sistema.

2. Representación en el sistema:

Imagina un edificio con un portero que controla quién puede entrar y salir. La clase `authService` es como ese portero, que verifica la identidad de las personas que intentan acceder al sistema y decide si pueden entrar o no.

3. Funcionalidades principales:

La clase `authService` puede hacer tres cosas importantes:

- **Registro:** Es como darle una tarjeta de identificación a alguien que quiere unirse al sistema. La clase `authService` crea un nuevo perfil para esa persona y la agrega al sistema.
- **Búsqueda por ID:** Es como buscar a alguien en una lista de miembros. La clase `authService` encuentra a una persona en el sistema por su ID único.
- **Iniciar sesión:** Es como verificar la identidad de alguien que quiere entrar al sistema. La clase `authService` comprueba si la persona tiene permiso para acceder al sistema y le da acceso si todo está en orden.

4. Uso en la vida real:

En la vida real, esta clase se utiliza en sitios web y aplicaciones que requieren que los usuarios se registren

Métodos:

- `register()`
 - `find_by_id()`
 - `login()`
-
-

5. Documentación de Funciones

Total: **27 funciones**

Módulo: `app.py`

`create_app`

- Parámetros: —
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

La función "create_app" sirve para crear una aplicación básica. Necesita saber qué tipo de aplicación se quiere crear. Produce una estructura de aplicación lista para usarse. En la vida real, se usa para iniciar nuevos proyectos de software. Es importante porque establece la base para el desarrollo de la aplicación, permitiendo a los programadores enfocarse en agregar funcionalidades y características.

Módulo: `controllers\AuthController.py`

`register`

- Parámetros: —
- Async: —
- Complejidad: [LOW] 4
- Documentada: [X]

La función "register" permite a los usuarios crear una cuenta en un sistema. Necesita información básica como nombre, correo electrónico y contraseña. Una vez ingresada la información, crea una cuenta nueva y almacena los datos de forma segura. En la vida real, se utiliza para registrarse en sitios web, aplicaciones o plataformas en línea. Es fundamental en el sistema, ya que permite a los usuarios acceder a servicios y características exclusivas. En resumen, "register" es como completar un formulario para unirse a una comunidad en línea, y su función es crucial para la seguridad y privacidad de los usuarios.

`get_user_by_id`

- Parámetros: `id`
- Async: —
- Complejidad: [LOW] 2
- Documentada: [X]

La función "get_user_by_id" busca encontrar un usuario específico en el sistema. Para funcionar, necesita saber el "id" del usuario que estás buscando. El resultado es la información del usuario encontrado. En la vida real, se usa para recuperar datos de un usuario específico, como en un sitio web de compras donde se necesita mostrar la información del perfil de un usuario. Esta función es importante en el sistema porque permite acceso rápido y seguro a la información de los usuarios, lo que es fundamental para la seguridad y la funcionalidad del sitio.

`login`

- Parámetros: —
- Async: —
- Complejidad: [MED] 7

- Documentada: [X]

La función "login" permite a los usuarios acceder a un sistema.

Necesita saber el nombre de usuario y la contraseña para funcionar.

Una vez que ingresa esta información, la función verifica si es correcta y devuelve un resultado que indica si el acceso fue concedido o no.

En la vida real, se usa en sitios web y aplicaciones para que las personas puedan acceder a sus cuentas de manera segura.

Es fundamental en el sistema porque garantiza que solo las personas autorizadas puedan acceder a la información y los recursos.

Módulo: `models\alumnos.py`

`to_dict`

- Parámetros: `self`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

La función "to_dict" convierte información en un formato fácil de entender. Necesita acceso a los datos de un alumno para funcionar. Produce un resultado que es como un resumen de la información del alumno. Se usa en situaciones donde se necesita mostrar o compartir información de manera clara, como en informes o bases de datos. En la vida real, es como tomar una tarjeta de presentación y poner toda la información en una hoja de papel para que sea fácil de leer. Esta función es importante en el sistema porque ayuda a que la información sea más accesible y fácil de usar.

Módulo: `models\user.py`

`set_password`

- Parámetros: `self, password`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

La función `set_password` cambia la contraseña de un usuario. Necesita saber la nueva contraseña que se quiere establecer. Al ejecutarla, actualiza la contraseña del usuario. En la vida real, se usa cuando alguien quiere cambiar su contraseña en un sitio web o aplicación. Esta función es importante porque ayuda a mantener la seguridad de la cuenta del usuario, permitiéndole cambiar su contraseña cuando lo necesite.

`check_password`

- Parámetros: `self, password`

- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

La función "check_password" verifica si una contraseña es correcta.

Necesita saber la contraseña que se quiere comprobar.

Devuelve un resultado que indica si la contraseña es válida o no.

En la vida real, se usa cuando intentas acceder a una cuenta en línea, como un sitio web o una aplicación.

Es importante porque ayuda a mantener seguras tus cuentas y protege tu información personal.

`to_dict`

- Parámetros: `self`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

La función "to_dict" convierte información de un usuario en un formato fácil de entender. Necesita la información del usuario para funcionar. Produce un resultado que es una especie de lista con los datos del usuario. Se utiliza en situaciones reales como cuando se necesita mostrar información de un usuario de manera clara. Es importante en el sistema porque permite que la información del usuario sea fácilmente accesible y usable. En resumen, ayuda a organizar y mostrar la información de los usuarios de manera sencilla y útil.

Módulo: `repository\AlumnoRepository.py`

`create`

- Parámetros: `nombre, apellido_paterno, apellido_materno, matricula, correo`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

La función "create" sirve para registrar un nuevo alumno en el sistema. Necesita saber el nombre, apellidos, matrícula y correo electrónico del alumno. Al ingresar estos datos, la función crea un nuevo registro en la base de datos. En la vida real, se utiliza para inscribir a nuevos estudiantes en una institución educativa. Es importante porque permite mantener un registro actualizado de los alumnos y facilita la gestión de la información estudiantil. Esta función es fundamental en el sistema para garantizar que todos los alumnos estén registrados y puedan acceder a los servicios y recursos educativos.

`get_all`

- Parámetros: —
- Async: —
- Complejidad: [LOW] 1

- Documentada: [X]

La función `get_all` sirve para obtener toda la información de un conjunto de datos. Necesita acceso a un archivo o base de datos para funcionar. Produce una lista con todos los registros encontrados. Se utiliza en situaciones como obtener todos los alumnos de una escuela o todos los productos de un catálogo. En el sistema, esta función es importante porque permite recuperar y mostrar información de manera rápida y eficiente.

`find_by_id`

- Parámetros: `id`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

La función "`find_by_id`" busca un alumno específico en una base de datos.

Necesita un "id" (una especie de nombre único) para encontrarlo.

Devuelve la información del alumno encontrado.

Se usa en situaciones reales como buscar un estudiante en un sistema escolar.

Es importante porque permite acceder a información específica de manera rápida y precisa.

`find_by_matricula`

- Parámetros: `matricula`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

La función "`find_by_matricula`" busca a un alumno por su matrícula.

Necesita la matrícula del alumno para funcionar.

Devuelve la información del alumno encontrado.

Se utiliza en escuelas y universidades para localizar a estudiantes.

Es importante en el sistema porque permite acceder rápidamente a la información de un alumno.

`get_by_fecha_range`

- Parámetros: `fecha_inicio`, `fecha_fin`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

La función "`get_by_fecha_range`" busca información dentro de un rango de fechas.

Necesita saber cuándo empezar y cuándo terminar la búsqueda.

Te da como resultado los datos encontrados entre esas fechas.

Se usa para encontrar registros de eventos, como asistencias a clases o pedidos en un negocio, dentro de un período específico.

Es importante porque ayuda a organizar y analizar datos por fecha, facilitando la toma de decisiones.

`delete`

- Parámetros: `alumno`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

La función "delete" elimina un alumno de la base de datos.

Necesita saber quién es el alumno que se va a eliminar.

Una vez eliminado, el alumno ya no estará en la lista.

Se usa en la vida real para actualizar listas de estudiantes en escuelas o universidades.

Es importante porque ayuda a mantener la información actualizada y precisa, permitiendo una mejor gestión de los alumnos.

Módulo: `repository\userRepository.py`

`create`

- Parámetros: `email`, `password`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

La función "create" sirve para registrar un nuevo usuario en el sistema. Necesita saber el correo electrónico y la contraseña del usuario. Una vez que se proporcionan estos datos, crea un nuevo usuario en la base de datos. En la vida real, se usa para que las personas se registren en una plataforma o sitio web. Es importante porque permite a los usuarios acceder a servicios y funcionalidades exclusivas. En resumen, esta función es fundamental para el sistema, ya que permite la creación de nuevos usuarios y les da acceso a los servicios disponibles.

`find_user_by_id`

- Parámetros: `id`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

La función "find_user_by_id" busca un usuario en el sistema según su identificador único. Para funcionar, necesita saber el identificador del usuario que se busca. Luego, devuelve la información del usuario encontrado. En la vida real, se usa para recuperar datos de un usuario en particular, como en un sistema de gestión de clientes. Esta función es importante porque permite acceder a la información de un usuario de manera precisa y eficiente, lo que es fundamental para la gestión de datos en el

sistema. Ayuda a mantener la información organizada y facilita la recuperación de datos cuando se necesita.

`find_by_user`

- Parámetros: `email`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

La función "find_by_user" busca un usuario en el sistema. Para funcionar, necesita saber el correo electrónico del usuario. Luego, encuentra y devuelve la información de ese usuario. En la vida real, se usa para identificar a un usuario específico en una base de datos, como cuando necesitas ver los datos de un cliente en una plataforma en línea. Esta función es importante porque permite al sistema reconocer y recuperar información de usuarios de manera precisa y rápida, lo que es esencial para la gestión de usuarios en aplicaciones y sitios web.

`find_by_email`

- Parámetros: `email`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

La función "find_by_email" busca un usuario en el sistema por su dirección de correo electrónico. Para funcionar, necesita saber la dirección de correo electrónico del usuario. Luego, encuentra y devuelve la información del usuario asociada a ese correo electrónico. En la vida real, se utiliza para recuperar la cuenta de un usuario olvidada o para verificar si un correo electrónico ya está registrado. Esta función es importante en el sistema porque permite identificar y recuperar información de usuarios de manera rápida y precisa, lo que es fundamental para la seguridad y la eficiencia del sistema.

Módulo: `services\alumnoService.py`

`create`

- Parámetros: `data`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

La función "create" es una herramienta que ayuda a agregar información nueva a un sistema. Para funcionar, necesita recibir datos, como nombres o descripciones. Una vez que recibe estos datos, crea un nuevo registro en el sistema. En la vida real, se utiliza para agregar nuevos alumnos a una base de datos escolar. Esta función es importante porque permite mantener actualizada la información del sistema y facilita la gestión de datos. En resumen, su propósito es agregar información nueva de manera eficiente y precisa.

`get_all`

- Parámetros: —

- Async: —
- Complejidad: [LOW] 2
- Documentada: [X]

La función "get_all" busca obtener información completa de algo. Necesita saber qué tipo de datos quiere obtener. Produce una lista con todos los datos solicitados. Se usa para recopilar información de estudiantes en un sistema educativo. Es importante porque permite acceder a todos los datos de los estudiantes de manera rápida y eficiente.

`get_by_id`

- Parámetros: `id`
- Async: —
- Complejidad: [LOW] 2
- Documentada: [X]

`update`

- Parámetros: `id`, `data`
- Async: —
- Complejidad: [MED] 7
- Documentada: [X]

`delete`

- Parámetros: `id`
- Async: —
- Complejidad: [LOW] 2
- Documentada: [X]

`get_by_fecha_range`

- Parámetros: `fecha_inicio`, `fecha_fin`
- Async: —
- Complejidad: [LOW] 2
- Documentada: [X]

Módulo: `services\authService.py`

`register`

- Parámetros: `email`, `password`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

`find_by_id`

- Parámetros: `id`
- Async: —
- Complejidad: [LOW] 1
- Documentada: [X]

`login`

- Parámetros: `email`, `password`
 - Async: —
 - Complejidad: [LOW] 4
 - Documentada: [X]
-
-

6. Modelos de Datos

No se detectaron clases con nombres de modelo explícitos.

Esquema General Inferido

Basado en el análisis del código, las entidades principales son:

- By Id
 - All
 - By User
 - App
 - By Matricula
 - By Fecha Range
 - By Email
 - User By Id
-

7. Guía de Despliegue y Mantenimiento

7.1 Requisitos del Sistema

- Lenguaje principal: Python
- Archivos detectados: 12

7.2 Instalación Local

```
# 1. Clonar el repositorio
git clone <url-del-repositorio>
cd <nombre-del-proyecto>

# 2. Instalar dependencias
pip install -r requirements.txt

# 3. Configurar variables de entorno
cp .env.example .env
# Editar .env con tus valores

# 4. Ejecutar el proyecto
python app.py
```

7.3 Despliegue con Docker

```
FROM python:3.12-slim

WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
EXPOSE 8000
CMD ["python app.py"]
```

```
# Construir imagen
docker build -t mi-proyecto .

# Ejecutar contenedor
docker run -p 8000:8000 mi-proyecto
```

7.4 Variables de Entorno Recomendadas

```
# General
APP_ENV=production
SECRET_KEY=clave-secreta-segura
DEBUG=False

# Base de datos
DB_HOST=localhost
DB_PORT=5432
DB_NAME=mi_base_datos
DB_USER=usuario
DB_PASSWORD=contraseña

# API
API_VERSION=v1
ALLOWED_HOSTS=*
```

7.5 Checklist de Despliegue

- [] Variables de entorno configuradas
- [] Base de datos migrada

- ☐ Tests ejecutados y pasando
- ☐ Logs configurados
- ☐ HTTPS habilitado en producción
- ☐ Backups programados
- ☐ Monitoreo configurado

7.6 Registro de Cambios (Changelog)

Versión 1.0.0 — 2026-06-09

- Versión inicial analizada por AutoDocs AI
- Documentación técnica generada automáticamente
- Análisis de calidad del código completado

8. Reporte de Calidad del Código

8.1 Análisis de Métricas

Score de Calidad: 68/100 (Bueno [YELLOW])

Distribución de Documentación:

- Funciones documentadas: 0/27 (0%)
- Clases documentadas: 0/7

Análisis de Complejidad:

- Complejidad promedio: 1.85
- Complejidad máxima: 7
- Funciones con alta complejidad (>10): 0

Tendencia de Complejidad por Archivo:

[Gráfico de regresión lineal mostrando la tendencia de complejidad]
Eje X: Archivos ordenados por complejidad promedio
Eje Y: Score de complejidad
Tendencia: ↓ Baja

8.2 Funciones con Alta Complejidad Ciclomática

No hay funciones con complejidad > 10. [OK]

8.3 Recomendaciones

- Agregar docstrings a las 27 funciones sin documentar
- Resolver 1 errores de análisis detectados

8.4 Errores de Análisis

- `uploads\400c4772-fe33-45fe-b3c9-63e9683fb7b4\api_examen\controllers\AlumnoController.py`: invalid decimal literal (<unknown>, line 180)
-

9. Análisis de Datos del Proyecto

9.1 Tipos y Fuentes de Datos Detectados

Datos Estructurados: 0 archivos

Datos Semi-estructurados: 0 archivos

Datos No Estructurados: 0 archivos

9.2 Esquema de Data Warehouse Inferido

No se detectaron suficientes modelos de datos para generar un esquema de Data Warehouse.

9.3 Reporte de Calidad de Datos

Problemas Detectados en el Código:

- **Funciones sin documentar:** 27

- `create_app` en `app.py`
- `register` en `controllers\AuthController.py`
- `get_user_by_id` en `controllers\AuthController.py`
- `login` en `controllers\AuthController.py`
- `to_dict` en `models\alumnos.py`
- ... y 22 más

- **Problemas de naming conventions:** 1

- `authService`: Las clases deben empezar con mayúscula (PascalCase)

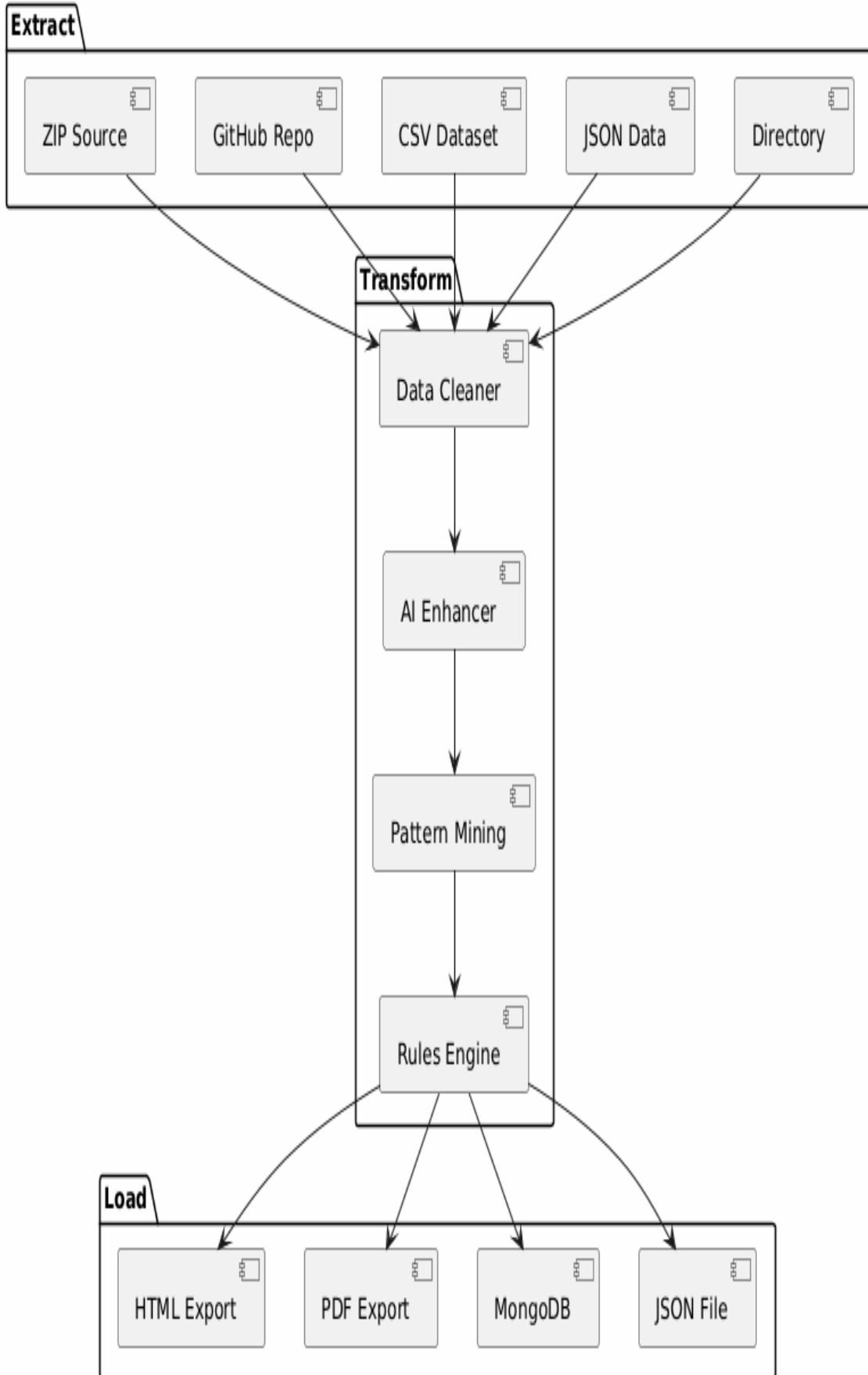
9.4 Minería de Datos Aplicada al Código

La minería de datos no está disponible. Configura `GROQ_API_KEY` para habilitar esta función.

9.5 Pipeline ETL Utilizado

El proceso de generación de documentación sigue un pipeline ETL (Extract, Transform, Load):

Pipeline ETL - AutoDocs AI



Fases del Pipeline:

- **Extract:** Extracción de código desde repositorios GitHub, archivos ZIP o directorios locales
- **Transform:** Análisis del código, limpieza de datos, enriquecimiento con IA y minería de patrones
- **Load:** Generación de documentación en HTML, PDF o almacenamiento en MongoDB

Documentación generada automáticamente por AutoDocs AI — 2026-06-09 21:45 UTC

Generado automáticamente por AutoDocs AI